

Light Instruments User Guide

Thomas Rögglä - CWI

August 5, 2026

Contents

1	Introduction	1
1.1	Overview	1
1.2	The Instruments	1
2	Getting Started	2
2.1	Connecting the Instruments	2
2.2	Connecting the LED Controllers	2
2.3	Setting up the Node Editor	2
3	Node Setups	3
3.1	Concepts	3
3.1.1	Interacting with the Editor	3
3.2	Setups for Debugging & Exploration	4
3.3	Device Filtering	4
3.4	Sending commands	6

1 Introduction

Welcome to the Light Instruments project. This guide will help you understand how to use and configure the various instruments and how to use the node editor to design lighting setups and have them light up the LED strips.

1.1 Overview

The project consists of several ESP32-S3 based instruments that communicate wirelessly via ESP-NOW, LED controllers that also communicate wirelessly, a receiver that plugs into your computer and a node-based editor to process the data coming from the instruments and transform it into commands for the LED controllers.

1.2 The Instruments

At this point in time, there exist the following instruments:

- *Key Instrument*: An instrument containing several buttons, which when pressed or released send a single signal about the button status to the receiver.
- *Maracas*: Sends a single signal to the receiver when movement around a specific axis is detected.
- *IMU Maracas*: Comes in the same form factor as the Maracas and works in the same fashion, but uses an IMU (inertial motion unit) to detect activity. This offers activity detection around any axis and better noise resistance.
- *Rainstick*: Contains an internal infrared barrier, which detects the passing of the percussive media inside the instrument through it and sends a corresponding signal to the receiver.

- *Touch Instrument*: Contains three touch surfaces, which when approached or touched by a hand sends a continuous signal identifying the amount of touch detected. Depending on initial calibration, the surfaces may not only react to touch, but also proximity of a hand.
- *Percussion Instrument*: The percussion instrument detects vibration using a piezoelectric element. When the surface of the instrument is struck using a mallet or hand and the detector detects the resulting vibration, a continuous signal is sent to the receiver. The sensitivity can be tuned by adjusting the 100 kOhm potentiometer on the circuit board.

2 Getting Started

First, plug the receiver into your computer using a USB cable. Then, you can start setting up the instruments and the LED controllers.

2.1 Connecting the Instruments

The instruments are powered by 1000 mAh Lithium cells. Open the battery compartments, locate the battery and battery connector. Plug the battery into the connector to activate it. The *Maracas* and *IMU Maracas* have a blue power switch located on the circuit port. In this case, the battery can remain plugged in at all times and the instrument can be turned on or off using the switch.

Now, observe the circuit board. An orange LED should blink twice immediately after gaining power. This signifies the start of the internal setup process. After at most 10 seconds, the same orange LED should turn on permanently, indicating that the instrument successfully discovered the receiver and is ready to transmit data. If the LED does not turn on, verify that the receiver is plugged into a USB port and the two are within wireless communication range. The communication range should be at about 200m with clear single of sight, but might drop to 20-40m with obstacles an in indoor environments. Note that communication efficiency is also affected in areas with increased wireless (namely WiFi) activity.

Note about the touch instrument: During boot, the touch instrument goes through a calibration procedure, adjusting the sensitivity of the touch surfaces based on environmental factors. Thus, when plugging in the instrument, try and avoid getting close to the touch surfaces with your hands or any other parts as this will influence the calibration procedure and lead to erroneous baseline calibration, affecting accurate touch detection during operation.

2.2 Connecting the LED Controllers

The setup procedure for the LED controllers is largely the same as for the instruments. The controllers can be either powered through the USB-C port or the 5V screw terminals on the back of the circuit board. When using the screw terminals, verify correct polarity and voltage before connecting to power.

Before connecting to power, also make sure the desired number of LED strips are connected to the four LED strip ports on the circuit board.

Analogous to the instruments, the onboard orange status LED will blink twice to indicate the start of the boot process. Then, after at most ten seconds, the LED will turn on permanently to indicate successful connection to the receiver. If this is not the case, make sure the receiver is plugged in and within wireless communication range.

2.3 Setting up the Node Editor

In order to read data from the instruments and design your own light setups, to be displayed using the LED controllers, after plugging the receiver into your computer, start the *AMPLIFY Light Instrument Editor*. In the default setup, you will see two nodes on the main canvas: *Serial Input* and *Serial Output*.

To get started, in the *Serial Input* node, select the name of the receiver from the dropdown and press *Connect*. On macOS this will be something like `/dev/cu.usbmodem101`, while on Windows it will be something like `COM1`. Upon successful connection, the *Serial Output* node should display the string `Connected`.

For a quick initial test of the LED controllers, find the command bar at the bottom of the window. From the command dropdown select the command **rainbow** and hit the *Send* button. Now all LED strips should display an animated rainbow pattern. Select the command **stop** and hit *Send* to stop the animation.

You can also verify presence and active communication with the controllers and instruments by locating the activity panel in the top left. This will display the names of all devices which the receiver was able to communicate, separated between instruments and LED controllers. Note that it might take a few seconds for this panel to be displayed. Names of devices from which no data was received recently will slowly fade out.

3 Node Setups

3.1 Concepts

The Light Instrument Editor uses a node-based interface to design lighting behaviors. Instead of writing code, you create a “node setup” (also known as a graph or patch) by placing functional blocks on a canvas and connecting them with lines.

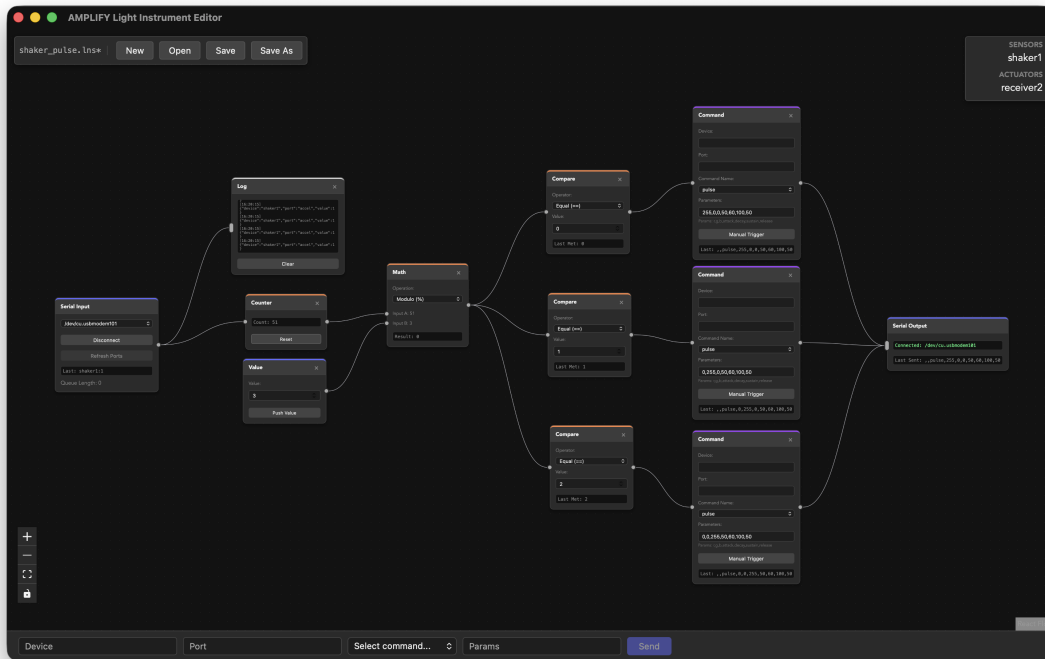


Figure 1: A basic node setup showing data flow from a Serial Input node to a Serial Output node via several Processing nodes.

Data flows from left to right: it starts at an **Input** node (like the physical serial port or a file simulation), passes through various **Processing** or **Math & Logic** nodes that transform the signals, and finally reaches an **Output** node which sends commands to your LED controllers or logs data to a file. This visual approach allows you to quickly experiment with different sensor-to-light mappings without needing to recompile firmware.

3.1.1 Interacting with the Editor

To build your setup, you can interact with the editor using the following commands:

- **Adding Nodes:** Press **Shift+A** to open the search menu. Type the name of the node you want to add and press **Enter**, or click on the desired node in the list.

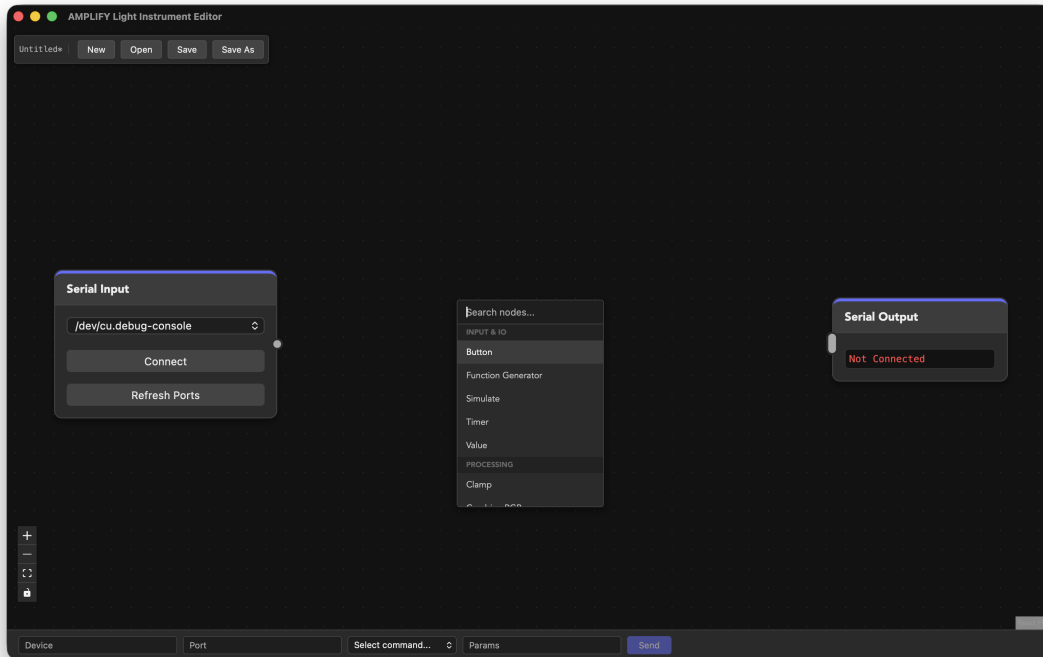


Figure 2: The context menu for adding nodes is activated by pressing Shift+A

- **Deleting Nodes:** Click the “×” button in the top-right corner of the node’s header.
- **Creating Connections:** Click and drag from an output handle (the circular socket on the right side of a node) to an input handle (on the left side of another node).
- **Deleting Connections:** Click on the connection line to select it, then press **Backspace** or **Delete** on your keyboard.

You can route the output of any node to the input of as many nodes as required. Most nodes only accept a single incoming connection. Some nodes, however, accept multiple inputs, e.g. the *Log* node or the *Serial Output* node. This is indicated by an elongated input socket.

3.2 Setups for Debugging & Exploration

When building a new setup or troubleshooting a sensor, it can be useful to visualize the raw data arriving from the instruments. The editor provides two primary nodes for this purpose:

- **Log Node:** Displays a scrollable history of every data packet received on its input, complete with device names, ports, and timestamps. It is useful for verifying that a device is sending data at the expected rate.
- **Graph Node:** Renders a real-time line chart of incoming numeric values. This is invaluable for visualizing signal noise, verifying calibration and tuning thresholds for logic nodes like *Peak Detection* or *Compare*.

A common debugging pattern is to branch your signal and connect it to a *Graph* or *Log* node in parallel with your main processing logic.

3.3 Device Filtering

In a large setup with multiple instruments, the receiver’s serial stream will contain data from many different sources. To prevent signals from interfering with each other, you can use the **Device Filter** node to direct

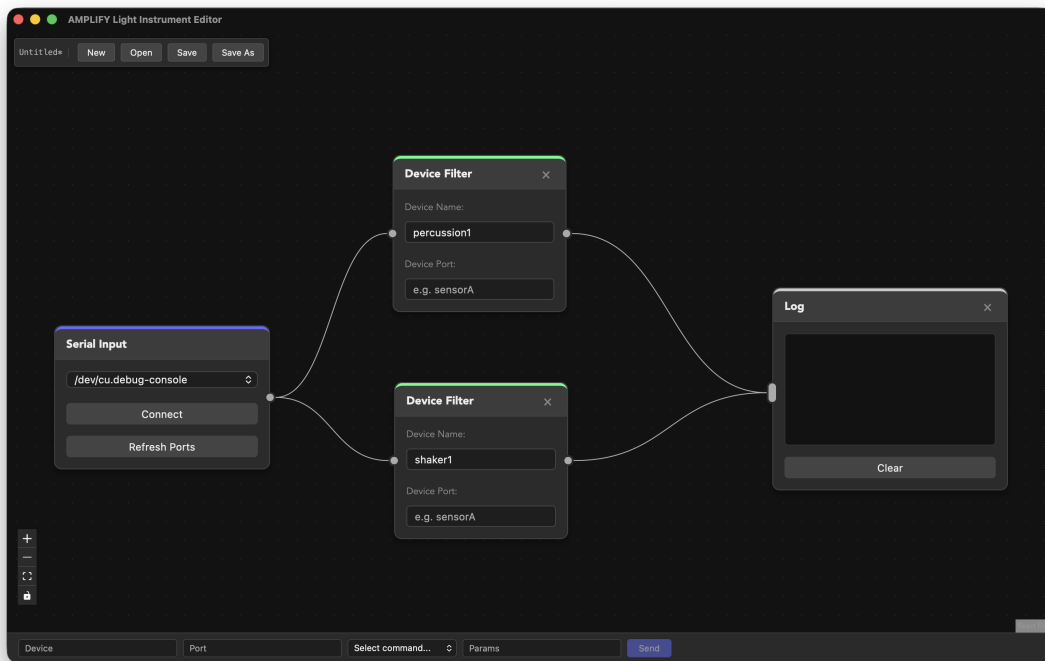


Figure 3: The Log node accepts multiple inputs, indicated by an elongated input socket



Figure 4: Debugging setup with all incoming messages being displayed in a Log node and a Graph node displaying the current rate of incoming messages per second through the use of the Rate node

data streams.

By configuring the *Device* and *Port* fields on this node, you can create a dedicated stream for a specific sensor. For example, if you only want to react to the “percussion1” instrument, you would set the device filter to “percussion1”. Only packets matching your filter will be passed through to the output handle, allowing you to build modular logic for each individual instrument in your setup.

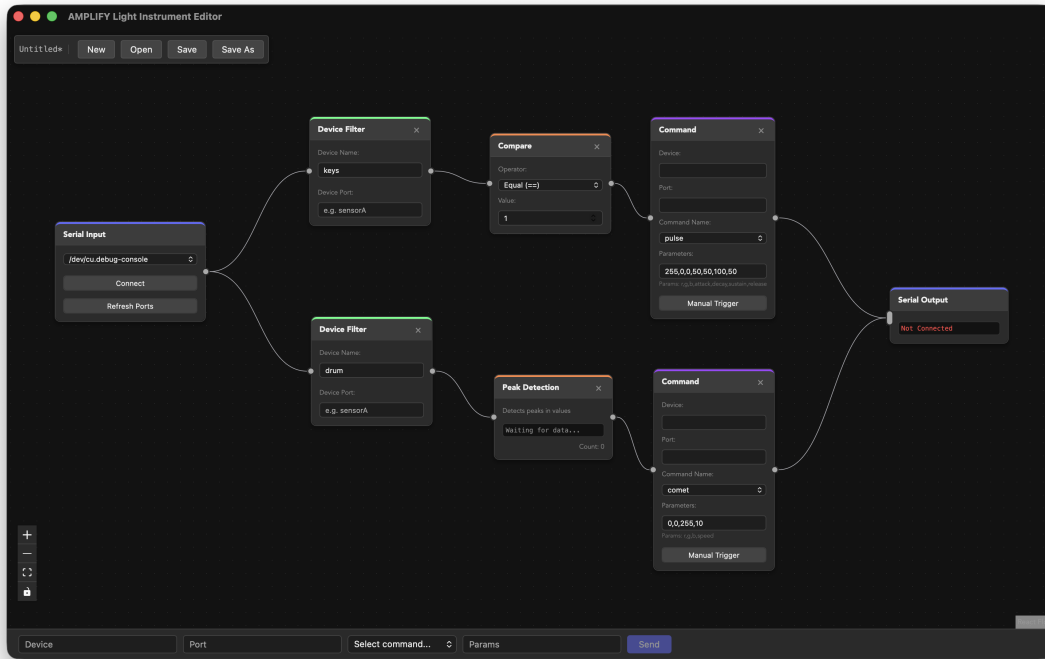


Figure 5: Two device filter nodes redirect the data streams in separate paths for the key instrument and the drum instrument, triggering different commands

3.4 Sending commands

To bridge the gap between sensor data and lighting effects, you use the **Command** node. This node acts as a translator: when it receives a trigger signal on its input, it emits a formatted command packet that the LED controllers understand. Note that the *Serial Output* node only accepts connections from a *Command* or *Script* node.

In the *Command* node, you can select from a range of animations and static effects (like **set**, **pulse**, or **rainbow**) defined in the controller firmware. You can specify a target device and port, or leave them blank to broadcast the command to all controllers.

To assist with configuration, a **parameter hint** appears below the input field whenever a command is selected, showing the expected format and required values (e.g., **r,g,b,speed**). You can enter static values directly into the parameter field. For dynamic effects, you can use the **#** character as a placeholder; this character will be automatically replaced by the numeric value of the incoming signal, allowing sensor intensity to directly control parameters like brightness, speed, or color.

For testing, commands can be triggered by pressing the *Manual Trigger* button on the corresponding *Command* node.